Lab 2 — second-site Supervisor deployment

Instantiates the root module for the second lab: **different vCenter, isolated network**, with the subnets swapped relative to lab 1:

Network	Port group	CIDR
Workload	`VM Network`	`192.168.1.0/24`
Management	`outer-mgmt-net`	`192.168.2.0/24`

Nested ESXi hosts already running at `192.168.1.241-243`. This example has its **own Terraform state** — nothing here touches lab 1 (`examples/lab`), and generated artifacts (TLS cert/key, enable spec) land in `examples/lab2/generated/`, separate from lab 1's.

1. Values you MUST update (search `CHANGE-ME` in `main.tf`)

Variable	What to set it to
`vcenter_server`	The lab-2 vCSA's system name (PNID) — FQDN **or IP**, whichever it was deployed with (the CN of its TLS cert tells you). If an FQDN: it must resolve to the vCSA's IP from this machine — verify with `dscacheutil -q host -a name <fqdn>` (macOS), not just `dig`. If an IP: use it as-is; this also sidesteps DNS-hijack risk entirely
`datacenter`	Datacenter name in the lab-2 vCenter (default `Datacenter`)
`physical_host_name`	The physical host's **inventory name** in vCenter — typically its IP
`physical_host_cluster`	Cluster containing the physical host (default `Cluster`)
`supervisor_cluster`	Cluster containing the three nested hosts (default `Supervisor-Cluster`)
`outer_datastore`	Datastore on the physical host where the HAProxy + NFS VMs deploy (default `datastore1`)

Variable	What to set it to
`management_gateway` / `management_dns`	Router + DNS reachable from `192.168.2.0/24` at this site
`workload_gateway` / `workload_dns`	Router + DNS reachable from `192.168.1.0/24` at this site

Verify-but-probably-keep (defaults follow the lab-1 conventions):

Variable	Default	Check
`nested_esxi_hosts`	`192.168.1.241-243`	must equal the hosts' vCenter **inventory names** exactly
`management_cp_starting_ip`	`192.168.2.231`	5 consecutive IPs free?
`nested_host_mgmt_ips` (in the module block)	`.1.24x → .2.24x`	mgmt IPs free? keys match `nested_esxi_hosts`?
`workload_ip_range`	`192.168.1.201-230`	outside DHCP, doesn't overlap the router/hosts/VMs
`vip_pool` / `vip_pool_usable`	`192.168.1.248/29` → `.249-.254`	free, outside DHCP
`nfs_ip` / `haproxy_ip` (module block / variable)	`.1.244` / `.1.245`	free
`ntp_servers` (module block)	`162.159.200.1`	reachable on UDP 123 from the physical host

Secrets go in `secrets.auto.tfvars` (never committed — see step 3).

---

2. Prerequisites before `terraform apply`

On the nested ESXi hosts (do this FIRST):

- [ ] **Each nested ESXi VM exposes hardware virtualization (VHV).** Edit Settings → expand the **CPU** row (caret on the left) → check **"Hardware virtualization: Expose hardware assisted virtualization to the guest OS"**. VM must be powered off (the row hides/greys while running). If the checkbox isn't there, set the flag directly: `govc`

`vm.change -vm <vm-path> -nested-hv-enabled=true`, or VM Options → Advanced → Configuration Parameters → add `vhv.enable = TRUE`. Verify from inside the host: `esxcli hardware cpu global get` → **HV Support: 3**. Without it the host boots and joins vCenter normally, but **every VM it tries to power on fails** with *"This host does not support Intel VT-x … VHV disabled"* — first the vCLS VMs, later the Supervisor control plane. (Terraform-built hosts get this automatically via `nested_hv_enabled = true` in the nested-esxi module; hand-built VMs must set it manually.) Tip: do it in the same power-off window as the vNIC additions below.

- [ ] **Set a unique hostname on each host** — see §2a below for the exact commands. ESXi defaults to `localhost`; with that, every spherelet receives the same client-cert identity (`system:node:localhost`) and Kubernetes blocks node registration — the cluster comes up with zero worker nodes. See `TROUBLESHOOTING.md` → "Supervisor ESXi nodes never join".

- [ ] Each nested ESXi VM has **three network adapters**, in this exact order (vmnic numbering follows adapter order, and the module pins DVS uplinks by vmnic name — get the order wrong and Supervisor traffic exits the wrong subnets):

Adapter	Becomes	Port group	Role
1	vmnic0	`VM Network`	host management (vmk0)
2	vmnic1	`VM Network`	DVS uplink1 → workload
3	vmnic2	`outer-mgmt-net`	DVS uplink2 → management

To add/fix via the vSphere Client, per nested VM:

1. Right-click the VM → **Edit Settings**.
2. If an existing adapter 2 points at `outer-mgmt-net`, **change it to** `VM Network` (safe while nothing claims vmnic1 — no vmk, no DVS yet).
3. **Add New Device** → **Network Adapter** → port group `outer-mgmt-net`, type **VMXNET 3**, "Connect At Power On" checked → OK.
4. **Full power-cycle the VM** — a guest reboot does NOT reveal hot-added NICs (ESXi skips the PCI rescan; runbook Root Cause #7): host → Maintenance Mode → Enter, VM → Power Off → Power On, wait for the host to reconnect, exit Maintenance Mode. One host at a time.
5. **Verify the mapping by MAC** (the host can't show outer port groups): Edit Settings lists each adapter's MAC; compare with host → Configure → Networking → Physical adapters (or `govc host.esxcli -host=<host-path>`

`network nic list`). vmnic1's MAC must be the `VM Network` adapter, vmnic2's the `outer-mgmt-net` one.

- [ ] Expected host networking at this stage (naming trap): the host's own **"Management Network"** (`vmk0` on `vSwitch0`/vmnic0) lives on the **workload** CIDR — `192.168.1.24x` — because that's the host's address. The `192.168.2.0/24` "management" subnet is management for the *Supervisor control plane*, not the hosts; the host only gets a leg on it when Terraform adds `vmk1` (sup-host-mgmt). Leave `vmk0` where it is, and leave vmnic1/vmnic2 unattached — the DVS claims them.

- [ ] All three hosts joined to the Supervisor cluster and `connected`.

- [ ] Cluster vLCM image matches the installed ESXi build (vSphere UI → Cluster → Updates → Image → "All hosts compliant"). Mismatch fails Supervisor enable with VIB download errors.

In the lab-2 vCenter:

- [ ] Datacenter, both clusters, physical host, and datastore exist under the names configured above. **Terraform creates none of these** — every `datacenter` / `*_cluster` / `*_host` / `*_datastore` variable is a lookup of existing inventory, and `plan` fails if a name doesn't match. In particular, create the Supervisor cluster and join the three nested hosts to it by hand (vSphere Client → New Cluster → drag hosts in, or `govc cluster.create` + `govc cluster.add`).

- [ ] Port groups `VM Network` and `outer-mgmt-net` exist on the physical host (Terraform sets their security flags — you don't need to).

- [ ] SSO admin credentials for the account in `secrets.auto.tfvars`.

Network / environment:

- [ ] Routing between `192.168.1.0/24` and `192.168.2.0/24` (the Supervisor enable validates cross-subnet DNS from the CP VM).

- [ ] All static IPs in §1 are outside any DHCP scope.

- [ ] WAN egress from the workload subnet — the Ubuntu cloud OVA and the HAProxy Dataplane API binary download at deploy time.

- [ ] `vcenter_server` resolves correctly from this machine **and** will resolve correctly from the CP VMs (via `management_dns`). A public CNAME on the vCenter name will poison both — see `TROUBLESHOOTING.md` → DNS resolution.
- [ ] Clocks sane on the physical host and vCSA (Terraform enables NTP on the physical host, but a wildly wrong vCSA clock still breaks TLS).

On this machine (once per workstation):

- [ ] `make -C ../.. install-deps` — terraform, govc, python3+pyvmomi, curl, jq, openssl.

2a. Setting the ESXi hostnames (required — do before Terraform)

What to set: the recommended values for this lab are

Host	`--host`	`--domain`
192.168.1.241	`nested-esxi-1`	`lab2.skynetsystems.io`
192.168.1.242	`nested-esxi-2`	`lab2.skynetsystems.io`
192.168.1.243	`nested-esxi-3`	`lab2.skynetsystems.io`

The actual strings are your choice — the rules that matter:

- **Unique per host** (the whole point — `localhost` on all three is the failure mode).
- **Lowercase letters, digits, hyphens, dots only** — the FQDN becomes the Kubernetes node name, which must be DNS-1123 compliant (no underscores, no uppercase).
- **Stable** — spherelet's cert identity is minted from it; renaming later means re-issuing spherelet certs.
- **Avoid** `.local` as the domain (mDNS conflicts, especially from macOS).
- No DNS records are required for these names; the domain is effectively a label.

Pick whichever method fits:

Helper script (from the repo root; checks first, sets only when wrong, safe to re-run — needs SSH enabled on the hosts, see the SSH primer in §3):

```
cd ~/Repos/vcf9-supervisor-terraform
SSHPASS='<esxi root password>' ./scripts/sv-set-hostnames -d <lab2-domain> \
  192.168.1.241=nested-esxi-1 \
  192.168.1.242=nested-esxi-2 \
  192.168.1.243=nested-esxi-3
```

Manual over SSH, one host at a time:

```
ssh root@192.168.1.241 'esxcli system hostname set --host=nested-esxi-1 --
        domain=<lab2-domain>'
ssh root@192.168.1.241 'esxcli system hostname get'   # verify
```

ESXi Host Client (no SSH needed): browse to `https://<host-ip>/ui`, log in as root → **Networking** → **TCP/IP stacks** → **Default TCP/IP stack** → **Edit settings** → set Host name + Domain.

DCUI (VM console): F2 → log in → **Configure Management Network** → **DNS Configuration** → set Hostname.

Setting the hostname does **not** rename the host in vCenter inventory (that stays the IP it was added with) and does not disrupt the host.

If spherelet ever ran on a host while it was named `localhost`, also clear `/etc/vmware/spherelet/*.crt|*.key|*.pem` on it and restart spherelet after Supervisor enable (full recipe in TROUBLESHOOTING.md).

2b. Creating the Supervisor cluster (manual — Terraform won't do this)

Settings below are the lab-1 working configuration, read live from its cluster.

Via the vSphere Client

1. **Create the cluster:** right-click the datacenter → **New Cluster** → name it `Supervisor-Cluster` (or whatever you set `supervisor_cluster` to), with:

Setting	Value	Why
vSphere DRS	**On**, automation level **Fully Automated**	Supervisor requires DRS to place CP/pod VMs
vSphere HA	**On** (admission control on)	Supervisor requires HA

Setting	Value	Why
vSAN	**Off**	we use the NFS datastore instead

(Don't look for a "Manage all hosts with a single image" checkbox — that's the vSphere 7/8 wizard. In vSphere 9, image-based lifecycle management is the only mode, so there's nothing to tick; the image itself is configured after creation, next step.)

2. **Set the cluster image to the hosts' exact ESXi build** — do this after creating the cluster and BEFORE enabling Supervisor. Enable pushes the cluster image (with the spherelet VIBs) onto the hosts; any version mismatch fails with `Cannot download VIB` (runbook Root Causes #1–2).
    1. Import the offline depot matching the build the nested hosts actually run (e.g. `VMware-ESXi-9.0.x-<build>-depot.zip` from Broadcom support): **vSphere Client → Lifecycle Manager → Import → Bundle (.zip)**.
    2. **Cluster → Updates → Image → Edit → ESXi Version** dropdown → pick that exact build → **Validate** → **Save**.
    3. Target state: **"All hosts compliant."** Don't proceed until it is.
3. **Add the hosts:** right-click the cluster → **Add Hosts** → enter `192.168.1.241/.242/.243` with each host's `root` password → accept the thumbprints. (Order note: adding hosts before or after setting the image both work — compliance is evaluated once both exist.)

    The wizard leaves the hosts in maintenance mode — this is expected. Exit it on each host (right-click → Maintenance Mode → Exit, or):

    ```
    for h in 192.168.1.241 192.168.1.242 192.168.1.243; do
      govc host.maintenance.exit /Datacenter/host/Supervisor-Cluster/$h
    done
    ```

    Within ~60 s of leaving maintenance mode, vSphere auto-deploys two small **vCLS** VMs on the cluster — normal, DRS needs them, leave them alone. HA datastore/network alarms at this stage are also expected until step 4's advanced options are set and Terraform mounts `nfs-shared`.
4. **HA advanced options** (silences recurring HA alarms in a nested, single-datastore lab — set via Cluster → Configure → vSphere Availability → Edit → Advanced Options):

Option	Value	Why
`das.ignoreInsufficientHbDatastore`	`true`	HA wants ≥2 heartbeat datastores; nested hosts will only ever see `nfs-shared`
`das.ignoreRedundantNetWarning`	`true`	HA wants redundant management NICs; the nested hosts have one path

(See `TROUBLESHOOTING.md` → “HA alarm spam” for a pyvmomi snippet that sets these from the CLI — govc doesn’t expose them.)

Or via govc

```
# Point govc at the lab-2 vCenter:
export GOVC_URL=<lab2-vcenter-fqdn> GOVC_USERNAME=administrator@vsphere.local \
       GOVC_PASSWORD='<sso password>' GOVC_INSECURE=true

govc cluster.create -dc=Datacenter Supervisor-Cluster

govc cluster.change -drs-enabled -drs-mode=fullyAutomated -ha-enabled \
  /Datacenter/host/Supervisor-Cluster

for h in 192.168.1.241 192.168.1.242 192.168.1.243; do
  govc cluster.add -cluster=Supervisor-Cluster \
    -hostname=$h -username=root -password='<esxi root pw>' -noverify
done

# If any host was added in maintenance mode:
govc host.maintenance.exit /Datacenter/host/Supervisor-Cluster/192.168.1.241

# Verify: all three connected
govc find /Datacenter/host/Supervisor-Cluster -type h
```

The vLCM image assignment and the two HA advanced options still need the UI / pyvmomi as noted above.

If you hit “vSphere HA agent is not reachable from vCenter”

Common right after this phase (maintenance-mode exits, hostname changes, and fresh clocks all upset the FDM agent). In order:

1. Right-click the host → **Reconfigure for vSphere HA**; repeat per affected host. Fixes most cases — especially after the §2a hostname change.

2. Still broken → cluster → vSphere Availability → HA **off**, wait for the unconfigure tasks, HA **on** (clean agent reinstall).

3. Check clocks — HA's SSL sessions fail *silently* on skew, and hand-installed ESXi often ships with NTP off:

```
for h in 192.168.1.241 192.168.1.242 192.168.1.243; do
  govc host.date.info -host /Datacenter/host/Supervisor-Cluster/$h | grep
        -E 'Current|NTP'
  govc host.date.change -host /Datacenter/host/Supervisor-Cluster/$h -serv
        er 162.159.200.1
  GOVC_HOST=/Datacenter/host/Supervisor-Cluster/$h govc host.service
        enable ntpd
  GOVC_HOST=/Datacenter/host/Supervisor-Cluster/$h govc host.service
        start  ntpd
done
```

   Compare with the vCSA's own clock too (VAMI → Time).

4. Last resort: `ssh root@<host> '/etc/init.d/vmware-fdm restart'` and read `/var/log/fdm.log` — the error line names the cause.

(The *"insufficient heartbeat datastores"* warning is a different, expected alarm until Terraform mounts `nfs-shared` — step 4's advanced option silences it. The *agent not reachable* error, by contrast, must be resolved before Supervisor enable.)

Don't configure networking on the cluster beyond this — the `supervisor-dvs`, port groups, uplinks, and vmkernel NICs are all Terraform's job.

---

3. Commands to run (in order)

All paths relative to the repo root (`~/Repos/vcf9-supervisor-terraform`); the example lives in `examples/lab2/`.

> ***How to SSH into an ESXi host (step 0 needs this)***
>
> ***Account:*** *always the user* `root`*, with the ESXi* ***root password chosen when the host was installed*** *(this is a per-host password — not a vCenter/SSO login, and not the* `secrets.auto.tfvars` *passwords).*
>
> ***SSH is disabled by default on ESXi.*** *Enable it first, any one of these ways:*
>
> - ***vSphere Client:*** *select the host → Configure → System → Services → SSH → Start.*

- ***ESXi Host Client:*** *browse to* `https://<host-ip>/ui`*, log in as root → Manage → Services →* `TSM-SSH` *→ Start.*
- ***DCUI*** *(the yellow console screen on the VM): F2 → log in → Troubleshooting Options → Enable SSH.*
- ***govc*** *(from this machine, pointed at the lab-2 vCenter):* `GOVC_HOST=/<datacenter>/host/<cluster>/<host-ip> govc host.service start TSM-SSH`

Running a command over SSH *— either interactively:*

```
ssh root@192.168.1.241        # type the root password when prompted
esxcli system hostname get    # you're now in the ESXi shell
exit
```

or one-shot without opening a shell (this is the form the commands below use — everything inside the quotes runs on the host):

```
ssh root@192.168.1.241 'esxcli system hostname get'
```

When done, stop SSH again from the same menu (or `govc host.service stop TSM-SSH`*) — leaving it on triggers a persistent yellow warning on the host and is poor hygiene.*

```
# 0. FIRST: verify each nested host has a unique hostname — ESXi
#    installs default to "localhost", which silently breaks spherelet
#    node registration (§2 / TROUBLESHOOTING.md). The helper script
#    checks and only sets when needed (idempotent, safe to re-run):
cd ~/Repos/vcf9-supervisor-terraform
SSHPASS='<esxi root password>' ./scripts/sv-set-hostnames -d <lab2-domain> \
  192.168.1.241=nested-esxi-1 \
  192.168.1.242=nested-esxi-2 \
  192.168.1.243=nested-esxi-3
#    (Omit SSHPASS to be prompted per host. If SSH is disabled on a
#     host, enable it via govc against the lab-2 vCenter first:
#       GOVC_HOST=/<datacenter>/host/<supervisor-cluster>/<host-ip> \
#         govc host.service start TSM-SSH
#     — or use the DCUI/host client. Equivalent manual command:
#       ssh root@<host> 'esxcli system hostname set --host=<name> --
#        domain=<domain>')

cd ~/Repos/vcf9-supervisor-terraform/examples/lab2

# 1. Secrets file (gitignored) — copy the template and fill in:
cp secrets.auto.tfvars.example secrets.auto.tfvars
vi secrets.auto.tfvars          # vcenter_password, haproxy_password, vcenter_ip
chmod 600 secrets.auto.tfvars

# 2. Fill in the CHANGE-ME values:
vi main.tf
```

```
# 3. Initialize providers (first run only):
terraform init

# 4. Sanity-check the config:
terraform validate

# 5. Preview — expect ~20+ resources to create, 0 to destroy:
terraform plan

# 6. Apply (~25-30 min unattended; the Supervisor-enable step polls
#    up to 45 min at the end):
terraform apply

# 7. Post-apply outputs (kubectl login instructions, API VIP):
terraform output next_steps
```

Or drive it through the Makefile from the repo root:

```
cd ~/Repos/vcf9-supervisor-terraform
make TF_DIR=examples/lab2 init
make TF_DIR=examples/lab2 plan
make TF_DIR=examples/lab2 apply
```

(Note: `make hard-check` and the `scripts/sv-*` helpers are hardwired to lab 1's IPs/vCenter — don't use them against this site without editing.)

Teardown: `terraform destroy` from `examples/lab2/` — disables the Supervisor cleanly first (10–15 min), then removes the VMs, DVS, and policy.

---

4. After apply — verify

```
# All 4-6 nodes Ready (3 CP + your 3 ESXi agents). CP root password:
# ssh to the lab-2 vCSA → shell → /usr/lib/vmware-wcp/decryptK8Pwd.py
ssh root@<cp-floating-ip> 'kubectl get nodes'

# API reachable through the HAProxy VIP:
curl -sk --max-time 5 "$(cd examples/lab2 && terraform output -raw
        supervisor_api_vip)/version"
```

If the ESXi agent nodes are missing, work through `TROUBLESHOOTING.md` → "Supervisor ESXi nodes never join" — check the spherelet cert identity (hostname issue) first, then the vmk path.